

# NineDOF: An Absolute Orientation Sensor Fusion Algorithm (For Dummies?)

Kyle Qi

October 6, 2026

## 1 Introduction

This document accompanies the NineDOF PlatformIO based C++ library, a library used to calculate the absolute orientation of a device. The purpose of this document is to explain how the algorithm is designed from the ground up. No prior knowledge of controls, robotics, or sensor fusion is required. However a foundation in calculus, linear algebra and geometry is assumed. This document is written as a book that can be read chronologically, as opposed to a reference where a reader jumps to sections as necessary.

The algorithm uses an Extended Kalman Filter (EKF) to fuse accelerometer, gyroscope, and magnetometer to produce an orientation of the device in the global frame. Each sensor contributes three degrees of freedom, hence the name NineDOF. The underlying library solely handles fusion and assumes that it is being fed non-garbage values from the sensors in use. Therefore, discussions around denoising sensor readings will remain limited in this document.

## 2 The Kalman Filter

Dynamical systems such as cars, robots and drones primarily have two sources of truth available to them to observe their physical evolution over time: Their mathematical model comprised of physics equations, and sensors that measure their state empirically. Both methods have their own benefits and drawbacks:

- **Models** are explicit, robust to noise, and easy to simulate but can be inaccurate as they encode a lot of assumptions, typically ignoring or inaccurately capturing factors such as friction and drag. These non-modelled factors are called process noise.
- **Sensors** give empirical readings of the state of a system, capturing non-modelled effects, but produce non-differentiable and noisy signals. Sources of variation

Instead of using one approach to determine the state of a system, the Kalman Filter uses both the system model and the sensor readings to produce a more complete picture of the state of a system. Both models and sensor readings introduce uncertainty due to non-modelled effects and noise respectively, so the Kalman Filter uses both to optimally produce a more complete picture.

The Kalman Filter is comprised of two stages: The Predict Stage, in which mathematical models are used to predict what the future state of the system will be, and then the update stage where sensor readings are used to "correct" the update stage. At each timestep, the Kalman Filter produces two pieces of information useful for the consumer: A current state estimate and an estimate uncertainty matrix. Additionally, it produces a future prediction state and prediction uncertainty matrix for the next time step of the algorithm. Since the Kalman Filter only requires the previous state for its current calculation, it makes it computationally cheap and lightweight. No additional prior time knowledge of the system is required.

## **2.1 The Predict Stage**

## **2.2 The Update Stage**

## **2.3 The Extended Kalman Filter**

The Extended Kalman Filter extends the Kalman Filter to non-linear applications by linearizing the system around the point of interest.

This is accomplished by

## **3 Sensors**

## **4 Accelerometer**

## **5 Gyroscope**

### **5.1 Gimbal Lock**

### **5.2 Quaternions**

### **5.3 Body vs Global Coordinate Frame**

### **5.4 Quaternions**

## **6 Magnetometer**

### **6.1 Hard and Soft Iron Offsets**

## **7 Notation**

## **8 References**

TODO: To be formatted.

This document was developed with the help of the resources listed below. Additionally, generative AI was used in the creation of this document, however its use is minimized as much as possible.

<https://kalmanfilter.net/>